Proposed resolution for 2019 comment CA 112

Document number:	P2113R0
Date:	2020-02-13
Project:	ISO/IEC JTC 1/SC 22/WG 21/C++
Audience subgroup:	Core
Revises:	*None*

Reply-to: Hubert S.K. Tong <hubert.reinterpretcast@gmail.com>

Notes

Consistent with the direction from Belfast (and the proposed change submitted with CA 112), the property of “structural” partial ordering that it ignores the form of function parameters for which arguments are not provided on the call expression is not replicated in the consideration of constraints. The form of such function parameters are not ignored for the case of non-template (but templated) functions. There is also no precedent for handling the constraints that relate to such function parameters (e.g., if there are constraints that involve template parameters for either template that are without a deduced argument after the two-way deduction process) in a way that they are ignored.

This also resolves (with modification) comment US 120.

Proposed Wording

☐ Hide text inserted in-line ☐ Hide text deleted in-line

In relation to N4849, modify in subclause 13.7.6.1 [temp.over.link] paragraph 6:

> Two *template-head*s are *equivalent* if their *template-parameter-list*s have the same length, corresponding *template-parameter*s are equivalent and such that if either *template-parameter* is declared with a *type-constraint*, they are both declared with *type-constraint*s that are equivalent, and if either *template-head* has a *requires-clause*, they both have *requires-clause*s and the corresponding *constraint-expression*s are equivalent. Two *template-parameter*s are *equivalent* under the following conditions:
>
> — […]
> — if they declare non-type template parameters, they have equivalent types ignoring the use of *type-constraint*s for placeholder types, and
> — if […]~~, and~~
> — ~~if either is declared with a *type-constraint*, they both are, and the *type-constraint*s are equivalent~~.
>
> […]

In relation to N4849, modify in subclause 13.7.6.2 [temp.func.order] paragraph 2:

> Partial ordering selects which of two function templates is more specialized than the other by transforming each template in turn (see next paragraph) and performing

template argument deduction using the function type. The deduction process determines whether one of the templates is more specialized than the other. If so, the more specialized template is the one chosen by the partial ordering process. If both deductions succeed, the partial ordering selects the more constrained template (if one exists) as ~~described by the rules in [temp.constr.order]~~ determined below.

Add a new paragraph to subclause 13.7.6.2 [temp.func.order] after paragraph 4:

Using the transformed function template's function type, perform type deduction against the other template as described in [temp.deduct.partial].

[*Example*: …]

If deduction against the other template succeeds for both transformed templates, constraints can be considered as follows:

- — If their *template-parameter-list*s (possibly including *template-parameter*s invented for an abbreviated function template ([dcl.fct])) or function parameter lists differ in length, neither template is more specialized than the other.
- — Otherwise:
  - — If exactly one of the templates was considered by overload resolution via a rewritten candidate with reversed order of parameters:
    - — If, for either template, some of the *template-parameter*s are not deducible from their function parameters, neither template is more specialized than the other.
    - — If there is either no reordering or more than one reordering of the associated *template-parameter-list* such that the corresponding *template-parameter*s of the *template-parameter-list*s are equivalent and such that the function parameters that positionally correspond between the two templates are of the same type, neither template is more specialized than the other.
  - — Otherwise, if the corresponding *template-parameter*s of the *template-parameter-list*s are not equivalent ([temp.over.link]) or if the function parameters that positionally correspond between the two templates are not of the same type, neither template is more specialized than the other.
- — Otherwise, if the context in which the partial ordering is done is that of a call to a conversion function and the return types of the templates are not the same, then neither template is more specialized than the other.
- — Otherwise, if one template is more constrained than the other ([temp.constr.order]), the more constrained template is more specialized than the other.
- — Otherwise, neither template is more specialized than the other.

[*Example*:

```
template <typename> constexpr bool True = true;
template <typename T> concept C = True<T>;

void f(C auto &, auto &) = delete;
template <C Q> void f(Q &, C auto &);
```

```
    void g(struct A *ap, struct B *bp) {
      f(*ap, *bp); // OK: Can use different methods to produce template parameters
    }

    template <typename T, typename U> struct X {};

    template <typename T, C U, typename V>
    bool operator==(X<T, U>, V) = delete;
    template <C T, C U, C V>
    bool operator==(T, X<U, V>);

    void h() {
      X<void *, int>{} == 0; // OK: Correspondence of [T, U, V] and [U, V, T]
    }
```

—*end example*]